FULL STACK DEVELOPMENT LAB

Question Paper Sets 5 & 6 : Questions with Answers

SET 5

Q1. Write a program to transfer data over http protocol using http module.

Aim: To write a program to transfer data over http protocol using http module.

Program (1b.js):

```
var http = require('http');
const server = http.createServer((req, res) => {
  res.write(req.url);
  res.end();
});
server.listen(8082, () => {
  console.log("Server is running on port 8082");
});
```

Steps to run:

1. Save the file as 1b.js
2. Terminal: node 1b.js
3. Open browser: http://localhost:8082/hi how are you

Output:

```
Terminal : Server is running on port 8082
Browser  : /hi%20how%20are%20you
```

Q2. Write a program to understand the working of namespaces and modules.

Aim: To write a program to understand the working of namespaces and modules.
Namespaces are good for organizing code in one file. Modules are file-based and preferred for modern projects (Webpack, ESBuild etc.).

mathModule.ts (module - uses export)

```
export function multiply(a: number, b: number): number {
  return a * b;
}
export const PI: number = 3.14;
```

2e.ts (namespace + importing the module)

```
import { multiply, PI } from "./mathModule";

namespace MathUtils {
  export function add(a: number, b: number): number {
    return a + b;
  }
  export function subtract(a: number, b: number): number {
    return a - b;
  }
}

// Using the namespace
console.log("Namespace Add:", MathUtils.add(10, 5));
console.log("Namespace Subtract:", MathUtils.subtract(10, 5));

// Using the module
console.log("Module Multiply:", multiply(10, 5));
console.log("Module PI:", PI);
```

Steps to run:

1. npm.cmd install -g typescript
2. tsc.cmd 2e.ts
3. node 2e.js

Output:

```
Namespace Add: 15
Namespace Subtract: 5
Module Multiply: 50
Module PI: 3.14
```

Q3. Write a program to work with form data.

Aim: To write a program to work with form data.
Folder structure: exp4 / 4b.js and exp4 / views / form.ejs , result.ejs
Install: npm install express ejs

exp4/views/form.ejs

```
<!DOCTYPE html>
<html>
<head>
  <title>Contact Form</title>
</head>
<body>
  <h1>Contact Us</h1>
  <form action="/submit" method="POST">
    <label>Name:</label><br />
    <input type="text" name="name" required /><br /><br />
    <label>Email:</label><br />
    <input type="email" name="email" required /><br /><br />
    <label>Message:</label><br />
    <textarea name="message" required></textarea><br /><br />
    <button type="submit">Send</button>
  </form>
</body>
</html>
```

exp4/views/result.ejs

```
<html>
<head> <title>Submission Received</title> </head>
<body>
  <h1>Thank you for contacting us!</h1>
  <p><strong>Name:</strong> <%= name %></p>
  <p><strong>Email:</strong> <%= email %></p>
  <p><strong>Message:</strong> <%= message %></p>
</body>
</html>
```

exp4/4b.js

```
const express = require('express');
const app = express();
const port = 8081;

app.use(express.urlencoded({ extended: true }));

// Set the view engine to EJS
app.set('view engine', 'ejs');

// Route to render form
app.get('/', (req, res) => {
  res.render('form');
});

// Route to handle form submission
app.post('/submit', (req, res) => {
  const { name, email, message } = req.body;
  res.render('result', { name, email, message });
});

// Start the server
app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`);
});
```

Steps to run:

1. cd exp4
2. node 4b.js
3. Open http://localhost:8081 , fill the form and click Send

Output:

```
Terminal : Server running at http://localhost:8081
Page 1   : "Contact Us" form (Name, Email, Message, Send button)
Page 2   : (localhost:8081/submit)
```

```
Thank you for contacting us!
Name: Kottakota Ashok
Email: ashok@gmail.com
Message: Hi My name is Ashok
```

Q4. Write a program to add styles (CSS & Sass Styling) and display data.

Aim: To write a program to add styles (CSS & Sass Styling) and display data.

Setup (React + Vite):

1. npm create vite@latest (choose React -> JavaScript)
2. cd project-name and npm install
3. npm install sass (needed to compile .scss files)

src/App.jsx

```
import "./App.css";
import "./App.scss";

function App() {
  const students = ["Ravi", "Sita", "Kiran"];
  return (
    <div className="container">
      <h1 className="title">Student Details</h1>
      <ul className="data-list">
        {students.map((student) => (
          <li key={student}>{student}</li>
        ))}
      </ul>
    </div>
  );
}
export default App;
```

src/App.css

```
.container {
  text-align: center;
  background-color: #f9f9f9;
  padding: 20px;
}
.title {
  color: #2c3e50;
  font-size: 2rem;
}
```

src/App.scss

```
$list-color: #27ae60;

.data-list {
  list-style: none;
  padding: 0;
  li {
    background-color: #d5f5e3;
    margin: 8px 0;
    padding: 10px;
    border-radius: 6px;
    font-weight: bold;
    &:hover {
      background-color: $list-color;
      color: white;
      cursor: pointer;
    }
  }
}
```

Steps to run:

npm run dev -> open http://localhost:5173

Output:

```
Heading "Student Details" (colour #2c3e50) on a light grey container,
followed by a list of green-tinted rounded boxes:
   Ravi
   Sita
   Kiran
On mouse hover each box turns dark green (#27ae60) with white text.
```

Q5. Write MongoDB queries to perform CRUD operations on document using insert(), find(), update(), remove()

Aim: To write MongoDB queries to perform CRUD operations on document using insert(), find(), update(), remove() Open MongoDB Compass -> connect -> click >_ Open MongoDB shell (mongosh).

Create database and collection:

```
use collegeDB
db.createCollection("students")
```

1. CREATE (insert):

```
db.students.insertOne({ name: "Ravi", age: 20, department: "CSE", year: 3 })

db.students.insertMany([
  { name: "Sita",  age: 21, department: "CSE", year: 4 },
  { name: "Kiran", age: 20, department: "ECE", year: 3 }
])
```

2. READ (find):

```
db.students.find()                          // all documents
db.students.find({ department: "CSE" })     // only CSE students
```

3. UPDATE: change Ravi's year from 3 to 4

```
db.students.updateOne({ name: "Ravi" }, { $set: { year: 4 } })
```

4. DELETE (remove):

```
db.students.deleteOne({ name: "Ravi" })
db.students.deleteMany({ department: "ECE" })
```

Note: the older shell methods insert(), update() and remove() do the same jobs as insertOne/insertMany, updateOne/updateMany and deleteOne/deleteMany (they are deprecated in the newer mongosh).

Output:

```
db.students.find()
{ _id: ObjectId('6abbe204a27a4f58c3eeb611'), name: 'Ravi',  age: 20, department: 'CSE', year: 3 }
{ _id: ObjectId('6abbe363a27a4f58c3eeb613'), name: 'Sita',  age: 21, department: 'CSE', year: 4 }
{ _id: ObjectId('6abbe363a27a4f58c3eeb614'), name: 'Kiran', age: 20, department: 'ECE', year: 3 }

db.students.find({ department: "CSE" })      -> Ravi and Sita

db.students.updateOne(...)
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }
   (Ravi now shows year: 4)

db.students.deleteOne({ name: "Ravi" })
{ acknowledged: true, deletedCount: 1 }

db.students.deleteMany({ department: "ECE" })
{ acknowledged: true, deletedCount: 1 }      -> only Sita remains
```

SET 6

Q1. Create a text file src.txt and add the following content to it. (HTML, CSS, Javascript, Typescript, MongoDB, Express.js, React.js, Node.js)

Aim: To create a text file src.txt and add the following content to it. (HTML, CSS, Javascript, Typescript, MongoDB, Express.js, React.js, Node.js)

Program (1c.js):

```
var fs = require('fs');

var content = "HTML\nCSS\nJavaScript\nTypeScript\nMongoDB\nExpress.js\nReact.js\nNode.js";

fs.writeFile("src.txt", content, (err) => {
  if (err) throw err;
  console.log("src.txt has been created with the given content.");
});
```

Steps to run:

node 1c.js (then open src.txt)

Output:

```
Terminal : src.txt has been created with the given content.
src.txt  :
HTML
CSS
JavaScript
TypeScript
MongoDB
Express.js
React.js
Node.js
```

Q2. Write a program to understand generics with variables, functions and constraints

Aim: To write a program to understand generics with variables, functions and constraints.

Program (2f.ts):

```
// 1. Generic Variable
let genericValue: <T>(value: T) => T = function <T>(value: T): T {
  return value;
};
console.log("Generic Variable (number):", genericValue<number>(42));
console.log("Generic Variable (string):", genericValue<string>("Hello"));

// 2. Generic Function
function identity<T>(arg: T): T {
  return arg;
}
console.log("Generic Function (boolean):", identity<boolean>(true));
console.log("Generic Function (array):", identity<number[]>([1, 2, 3]));

// 3. Generic Function with Multiple Types
function pair<A, B>(first: A, second: B): [A, B] {
  return [first, second];
}
console.log("Generic Pair:", pair<string, number>("Age", 25));

// 4. Generic with Constraint
interface HasLength {
  length: number;
}
function printLength<T extends HasLength>(item: T): void {
  console.log("Length is:", item.length);
}
printLength("TypeScript");   // string has length
printLength([1, 2, 3, 4]);   // array has length
// printLength(123);         // Error: number doesn't have 'length'
```

```
// 5. Generic Class
class Box<T> {
  private _value: T;
  constructor(value: T) {
    this._value = value;
  }
  getValue(): T {
    return this._value;
  }
}
const stringBox = new Box<string>("Generic Box");
console.log("Box Value:", stringBox.getValue());
```

Steps to run:

tsc.cmd 2f.ts

node 2f.js

Output:

```
Generic Variable (number): 42
Generic Variable (string): Hello
Generic Function (boolean): true
Generic Function (array): [ 1, 2, 3 ]
Generic Pair: [ 'Age', 25 ]
Length is: 10
Length is: 4
Box Value: Generic Box
```

Q3. Write a program for session management using cookies and sessions.

Aim: To write a program for session management using cookies and sessions.
Install: npm init -y then npm install express express-session cookie-parser

Program (5a.js):

```
const express = require('express');
const session = require('express-session');
const cookieParser = require('cookie-parser');

const app = express();
const PORT = 8081;

// Middleware setup
app.use(cookieParser());
app.use(express.urlencoded({ extended: true }));

// Session setup
app.use(
  session({
    secret: 'mySecretKey123',      // Change to a secure key
    resave: false,
    saveUninitialized: true,
    cookie: { maxAge: 60000 },     // 1 minute
  })
);

// Home Route
app.get('/', (req, res) => {
  if (req.session.username) {
    res.send(`
      <h1>Welcome back, ${req.session.username}!</h1>
      <p><a href="/logout">Logout</a></p>
    `);
  } else {
    res.send(`
      <h1>Welcome Guest</h1>
      <form method="POST" action="/login">
        <input type="text" name="username" placeholder="Enter Username" required/>
        <button type="submit">Login</button>
      </form>
    `);
  }
});

// Login Route
app.post('/login', (req, res) => {
  const { username } = req.body;
  if (username) {
    req.session.username = username;
    res.cookie('username', username);
    res.redirect('/');
  } else {
    res.send('Login failed. No username provided.');
  }
});

// Logout Route
app.get('/logout', (req, res) => {
  res.clearCookie('username');
  req.session.destroy((err) => {
    if (err) {
      return res.send('Error logging out.');
    }
    res.redirect('/');
  });
});

// Server listening
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

Steps to run:

node 5a.js -> open http://localhost:8081

Output:

```
Terminal : Server is running at http://localhost:8081
Step 1   : "Welcome Guest"  + [Enter Username] [Login]
Step 2   : type Aditya and click Login
Step 3   : "Welcome back, Aditya!"  + Logout link
           (session + cookie stored for 1 minute; Logout clears both)
```

Q4. Write a program for responding to events.

Aim: To write a program for responding to events.

Program (src/App.jsx):

```
import "./App.css";
import { useState } from "react";

function App() {
  const [message, setMessage] = useState("Click the button");

  function handleClick() {
    setMessage("Button Clicked!");
  }

  return (
    <div>
      <h1>React Events</h1>
      <p>{message}</p>
      <button onClick={handleClick}>
        Click Me
      </button>
    </div>
  );
}
export default App;
```

Steps to run:

npm run dev -> open the Local URL shown (e.g. http://localhost:5173)

Output:

```
Before click :  React Events
                Click the button
                [ Click Me ]
After click  :  React Events
                Button Clicked!
                [ Click Me ]
```

Q5. Install MongoDB and configure ATLAS

Aim: To install MongoDB and configure ATLAS.

Part A : Install MongoDB Community Server (Windows)

1. Open the browser and search Google for MongoDB -> click MongoDB Community Download.
2. Select the Version, Platform = Windows, Package = msi and click Download.
3. Run the downloaded .msi file. On the Setup Wizard welcome screen click Next.
4. Tick 'I accept the terms in the License Agreement' and click Next.
5. Choose Setup Type : click Complete (installs all program features).
6. Service Configuration : tick 'Install MongoD as a Service' and select 'Run service as Network Service user'. Keep the default Service Name (MongoDB), Data Directory and Log Directory. Click Next.
7. Click Install (allow the permission prompt) and wait for the progress bar to finish.
8. Click Finish to complete the setup wizard.
9. Verify: open Command Prompt and run mongod --version (or open MongoDB Compass and connect to mongodb://localhost:27017).

Part B : Configure MongoDB ATLAS (cloud database)

1. Go to https://www.mongodb.com/atlas and click Sign Up / Try Free (or sign in with Google).
2. Create an Organization and a Project (e.g. FSD-Lab).
3. Click Build a Database -> choose the FREE M0 shared cluster -> select provider (AWS) and region -> give a Cluster Name -> Create.
4. Security -> Database Access : Add New Database User -> enter a username and password -> Add User.
5. Security -> Network Access : Add IP Address -> click 'Allow Access from Anywhere' (0.0.0.0/0) or 'Add Current IP Address' -> Confirm.
6. Go to Database -> click Connect on the cluster -> choose Compass (or Shell/Drivers) and copy the connection string:

```
mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/
```

7. Open MongoDB Compass -> paste the connection string (replace <password> with your password) -> click Connect.
8. Click Create Database -> Database Name: collegeDB, Collection Name: students -> Create Database.
9. Open collegeDB -> students -> Add Data -> Insert Document and insert records to check the Atlas connection works.

Output:

```
MongoDB Compass shows the Atlas cluster connected (green) with the databases
admin, local and the newly created  collegeDB > students  collection.
```